1. (2 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = 2n \cdot \log n$
   b) $f(n) = 3n^2 + 4 \cdot 3^n$

   Lösung: a) $f(n) \in O(n \cdot \log(n))$ b) $f(n) \in O(3^n)$

2. (2 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = 3n \cdot \log n + n$
   b) $f(n) = 5 + 6 + ... + n$

   Lösung: a) $f(n) \in O(n \cdot \log(n))$ b) $f(n) \in O(n^2)$

3. (4 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = n^2 + 2n \cdot \log n$
   b) $f(n) = 3n^2 \cdot \log n + 0.5 \cdot 2^n$
   c) $f(n) = \log(n^2)/3$
   d) $f(n) = \log(n) + n$

   Lösung: a) $f(n) \in O(n^2)$ b) $f(n) \in O(2^n)$ c) $f(n) \in O(\log n)$ d) $f(n) \in O(n)$

4. (3 Punkte) Ein Programm braucht für eine Eingabe der Größe $n = 1000$ eine Laufzeit von 2 Sekunden.

   (a) (2 Punkte) Schätze die Laufzeit für $n = 2000$ und $n = 10\,000$, wenn die Laufzeit in $O(n)$, $O(n^2)$ bzw. $O(n^3)$ liegt.

   (b) (1 Punkt) Ein anderes Programm mit einer Laufzeit in $O(2^n)$ braucht für $n = 30$ eine Sekunde. Wie lange braucht es ungefähr für $n = 40$?

   Lösung: a)

	$n = 2000$	$n = 10\,000$
$O(n)$	4 s	20 s
$O(n^2)$	8 s	200 s $\approx$ 3 min
$O(n^3)$	16 s	2000 s $\approx$ 33 min

   b) $2^{10} \approx 1000$, also etwa 1000 s $\approx$ 17 min.

5. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(n):
        if a[i] == 7:
            for j in range(n):
                for k in range(n):
                    print("hello")
```

   Lösung: best case: $O(n)$, worst case: $O(n^3)$

6. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(n):
        for j in range(n):
            if a[i] == 0:
                for k in range(n):
                    print("hello")
```

Lösung: best case: $O(n^2)$, worst case: $O(n^3)$

7. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(n-1):
        for j in range(i+1,n):
            if a[i] == a[j]:
                return True
    return False
```

Lösung: best case: $O(1)$, worst case: $O(n^2)$

8. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(10):
        for j in range(n):
            if a[j] == 42:
                return True
    return False
```

Lösung: best case: $O(1)$, worst case: $O(n)$

9. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    summe = 0
    for i in range(n):
        for j in range(n):
            summe = a[i]
            return summe
```

Lösung: best case: $O(1)$, worst case: $O(1)$

10. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    summe = 0
    for i in range(n):
        for j in range(n):
            if a[i] == 5:
                summe+= a[j]
    return summe
```

Lösung: best case: $O(n^2)$, worst case: $O(n^2)$

11. (3 Punkte) Gegeben ist die folgende Funktion:

```
def foo(n):
    for i in range(n):
        for j in range(i):
            print('*')
    for k in range(n):
        print('#')
```

(a) (1 Punkt) Wie oft werden `*` und `#` beim Aufruf `foo(5)` ausgegeben?

(b) (1 Punkt) Wie oft wird `*` beim Aufruf `foo(n)` ausgegeben?

(c) (1 Punkt) In welcher Komplexitätsklasse liegt die Laufzeit von `foo`?

Lösung:

a) `*`: $0+1+2+3+4 = 10$ mal, `#`: 5 mal

b) $0+1+\ldots+(n-1) = \frac{n\cdot(n-1)}{2}$ mal

c) $O(n^2)$: Die Schleifen stehen hintereinander, $O(n^2)+O(n)$ liegt in $O(n^2)$.

12. (2 Punkte) Gegeben ist die folgende Funktion:

```
def foo(n):
    zaehl = 0
    while n > 1:
        n = n // 2
        zaehl += 1
    return zaehl
```

(a) (1 Punkt) Was liefern `foo(1000)` und `foo(1000000)`?

(b) (1 Punkt) In welcher Komplexitätsklasse liegt die Laufzeit von `foo`?

Lösung: a) 9 und 19 (1000, 500, 250, 125, 62, 31, 15, 7, 3, 1: neunmal halbiert)

b) $O(\log n)$, denn n wird in jedem Durchlauf halbiert.

13. (3 Punkte) Gegeben ist die folgende Funktion:

```
def gemeinsame(a, b):
    ergebnis = []
    for x in a:
        if x in b:
            ergebnis.append(x)
    return ergebnis
```

(a) (1 Punkt) Die Listen `a` und `b` haben jeweils n Elemente. In welcher Komplexitätsklasse liegt die Laufzeit im worst case? Begründe.

(b) (2 Punkte) Ändere die Funktion so, dass die Laufzeit in $O(n)$ liegt.

Lösung: a) $O(n^2)$: `x in b` durchsucht die Liste `b`, das ist selbst eine Schleife mit bis zu n Schritten, und sie wird n-mal ausgeführt.

```
# b)
def gemeinsame(a, b):
    menge = set(b)              # O(n)
    ergebnis = []
    for x in a:
        if x in menge:          # O(1)
            ergebnis.append(x)
    return ergebnis
```